DATA ANALYST PROJECT

Case Study 1

Marketplace Revenue Leakage, Customer Economics & Commercial Decision-Making

A SQL case study on one marketplace transactions table of 900 order lines. Five business questions look at revenue leakage, customer concentration, discount dependency, acquisition cohorts and a discount-reduction scenario.

DATABASE
case_study_1
1 table, 900 rows

TABLE
marketplace_transactions
order-line transactions

ANALYSIS
5 business questions
bridge, customers, discounts, cohorts, scenario

SQL DIALECT
MySQL-style syntax
DATE_FORMAT, STR_TO_DATE (inferred)

CONCEPTS COVERED
CTEs · Window functions · CASE
LAG, NTILE, PERCENT_RANK, RANK

REVIEW
15 flags listed
items to verify before sharing

WHAT THIS DOCUMENT CONTAINS

The project set-up and data checks, five analysis questions with their full SQL, result tables, charts and short findings, and a closing review of items to verify. SQL is shown exactly as written; only layout was changed. Charts use only values from the result tables. This document describes patterns in one table and does not claim causes.

CONTENTS

Table of Contents

Eight parts: project set-up, data checks, five analysis questions and a review of open items.

01 **Project Overview** 3
Goal, metric definitions, the five questions and the techniques used.

02 **Data and Exploration Checks** 4
Table, columns, sample rows, counts, and duplicate, NULL and refund checks.

03 **Q1. Commercial performance bridge** 8
GMV to contribution proxy, month-over-month growth and the Yes/No flag.

04 **Q2. Customer concentration and economic dependency** 12
Top 1%, 5%, 10% shares, repeat purchase and large customers with negative contribution.

05 **Q3. Discount dependency and commercial quality** 15
Discount rates, commercial risk label, customer segments and weak high-value lines.

06 **Q4. Acquisition cohort quality** 21
Acquisition-month cohorts, contribution per order and repeat customer rate.

07 **Q5. Commercial downside scenario** 24
10% discount cut, break-even GMV decline and tolerance by category.

08 **Review Flags and Open Items** 27
Question coverage, 15 review flags and what changed in this edition.

HOW TO READ THIS DOCUMENT

SQL is shown exactly as written; only line breaks and indentation were tidied. Result tables were rebuilt from the result screenshots in the source, and charts use only those values. Items to check are marked with a flag number (F1, F2 …) and listed together in Section 08.

SECTION 01

Project Overview

One marketplace transactions table and five business questions about where revenue leaks, who the customers are, and what discounts really cost.

900	513	235	5	5	3
ORDER LINES	DISTINCT ORDERS	CUSTOMERS	CATEGORIES	CHANNELS	SEGMENTS

Counts are taken from the source queries: 900 rows, 513 orders (Q1), 235 customers (Q4), and the checks in Section 02. See flags F1, F6 and F7.

Goal

- Rebuild the commercial bridge from order value (GMV) to contribution, including discounts, refunds and fees.
- Measure how much value comes from the top 1%, 5% and 10% of customers, and how many customers come back.
- Check whether higher discounts go with lower contribution per order, by category, channel and segment.
- Compare acquisition cohorts and test a 10% cut in discount spend.
- Show every number next to its SQL, and list what still needs checking.

Metric definitions used in the queries

Metric	Definition as written in the source
GMV	(quantity × unit_selling_price) − discount_amount
Delivered GMV	GMV of rows where order_status = 'Delivered'
Contribution proxy	GMV − refund_amount − payment_fee − fulfilment_cost (the queries apply it to Delivered rows only)
Seller commission	((quantity × unit_selling_price) − discount_amount) × seller_commission_rate
Discount rate	100 × SUM(discount_amount) ÷ SUM(quantity × unit_list_price)
Refund rate / burden	100 × SUM(refund_amount) ÷ SUM((quantity × unit_selling_price) − discount_amount)
Repeat customer	A customer with 2 or more distinct delivered order_id values
Break-even GMV decline %	(10% × delivered discount_amount) ÷ delivered contribution × 100, as written in Q5

The five questions

#	Business question	SQL techniques	Main output
Q1	Commercial performance bridge and month-over-month growth	CTEs, SUM(CASE…), LAG, CASE flag	GMV to contribution totals
Q2	Customer concentration and economic dependency	CTEs, NTILE, PERCENT_RANK, subqueries	Top-customer shares, repeat rate
Q3	Discount dependency and commercial quality	Window AVG() OVER, RANK, PERCENT_RANK	Risk label by category, segment table
Q4	Acquisition cohort quality	DATE_FORMAT, STR_TO_DATE, nested subqueries	Cohort counts, repeat rate, contribution
Q5	Commercial downside scenario (10% discount cut)	Conditional SUM, ratio per category	Break-even GMV decline %

SECTION 02

Data and Exploration Checks

Table set-up, column list, row counts, category counts, and the duplicate, NULL and refund checks run before the main analysis.

Database and table

Item	Detail
Database	case_study_1
Table	marketplace_transactions (the only table)
Rows	900 (SELECT COUNT(*))
Dialect	MySQL-style syntax (SHOW TABLES, DATE_FORMAT, STR_TO_DATE); inferred from the queries
Currency	Not stated in the source; amounts are shown without a currency symbol

SET-UP

```
CREATE DATABASE case_study_1;
USE case_study_1;
SHOW TABLES;

SELECT *
FROM marketplace_transactions
LIMIT 10;
```

Columns

Column	Where it appears
transaction_id, order_id, customer_id, seller_id, product_category, order_timestamp, customer_acquisition_date, acquisition_channel, order_status, quantity, unit_list_price	Shown in the sample output (first 10 rows)
customer_segment, unit_selling_price, discount_amount, refund_amount, payment_fee, fulfilment_cost, seller_commission_rate	Used in the queries; not visible in the sample output

Data types are not shown in the source, so none are listed here.

Sample rows

transaction_id	order_id	customer_id	seller_id	product_category	order_timestamp	customer_acquisition_date	acquisition_channel	order_status	quantity	unit_list_price
T00001	O00411	C0094	S029	Sports	8/14/2025	7/6/2025	Affiliate	Delivered	2	412.77
T00002	O00614	C0006	S027	Home & Kitchen	2/12/2025	5/19/2025	Social	Delivered	1	437.00
T00003	O00040	C0097	S030	Sports	9/30/2025	1/29/2023	Paid Search	Returned	1	321.99
T00004	O00062	C0203	S002	Electronics	9/20/2025	8/25/2025	Referral	Delivered	1	135.62
T00005	O00163	C0048	S023	Electronics	4/7/2025	4/9/2024	Social	Delivered	1	1,064.24
T00006	O00640	C0234	S049	Home & Kitchen	12/20/2025	5/29/2023	Organic	Cancelled	2	1,510.14
T00007	O00203	C0086	S069	Beauty	12/17/2025	6/23/2024	Social	Delivered	1	469.51
T00008	O00060	C0073	S023	Beauty	3/10/2025	4/24/2024	Social	Cancelled	1	141.29
T00009	O00256	C0167	S005	Sports	3/28/2025	3/24/2024	Affiliate	Delivered	1	895.32
T00010	O00385	C0104	S023	Beauty	4/19/2025	10/30/2024	Social	Delivered	1	393.45

Exploration queries and results

Question asked of the data	Result
How many rows are in the table?	900
What are the oldest and newest order_timestamp values?	1/1/2025 and 9/9/2025 (as returned; see F2)
Are any transaction_id values duplicated?	No rows returned (no duplicates)
Are any discount, refund, fee or fulfilment values NULL?	0 NULL values in each of the four columns
How much refund sits in each order status?	Only Returned lines carry refunds: 37,945.29 on 76 lines

ROW COUNT AND DATE RANGE

```
-- *) COUNT TOTAL ROWS IN THE TABLE
SELECT COUNT(*) AS TOTAL_ROWS
FROM marketplace_transactions;

-- *) DATE RANGE: OLDEST AND NEWEST DATES IN ORDER_TIMESTAMP
SELECT MIN(ORDER_TIMESTAMP) AS OLDEST_DATE,
       MAX(ORDER_TIMESTAMP) AS NEWEST_DATE
FROM marketplace_transactions;
```

DISTINCT VALUES AND COUNTS

```
-- *) DISTINCT VALUES AND COUNTS FOR ORDER_STATUS, PRODUCT_CATEGORY,
-- ACQUISITION_CHANNEL AND CUSTOMER_SEGMENT
SELECT 'ORDER_STATUS' AS COL, ORDER_STATUS AS COL_VALUE, COUNT(*) AS CNT
FROM marketplace_transactions
GROUP BY ORDER_STATUS

UNION ALL

SELECT 'PRODUCT_CATEGORY', PRODUCT_CATEGORY, COUNT(*)
FROM marketplace_transactions GROUP BY PRODUCT_CATEGORY

UNION ALL

SELECT 'ACQUISITION_CHANNEL', ACQUISITION_CHANNEL, COUNT(*)
FROM marketplace_transactions GROUP BY ACQUISITION_CHANNEL

UNION ALL

SELECT 'CUSTOMER_SEGMENT', CUSTOMER_SEGMENT, COUNT(*)
FROM marketplace_transactions GROUP BY CUSTOMER_SEGMENT
ORDER BY COL, CNT DESC;
```

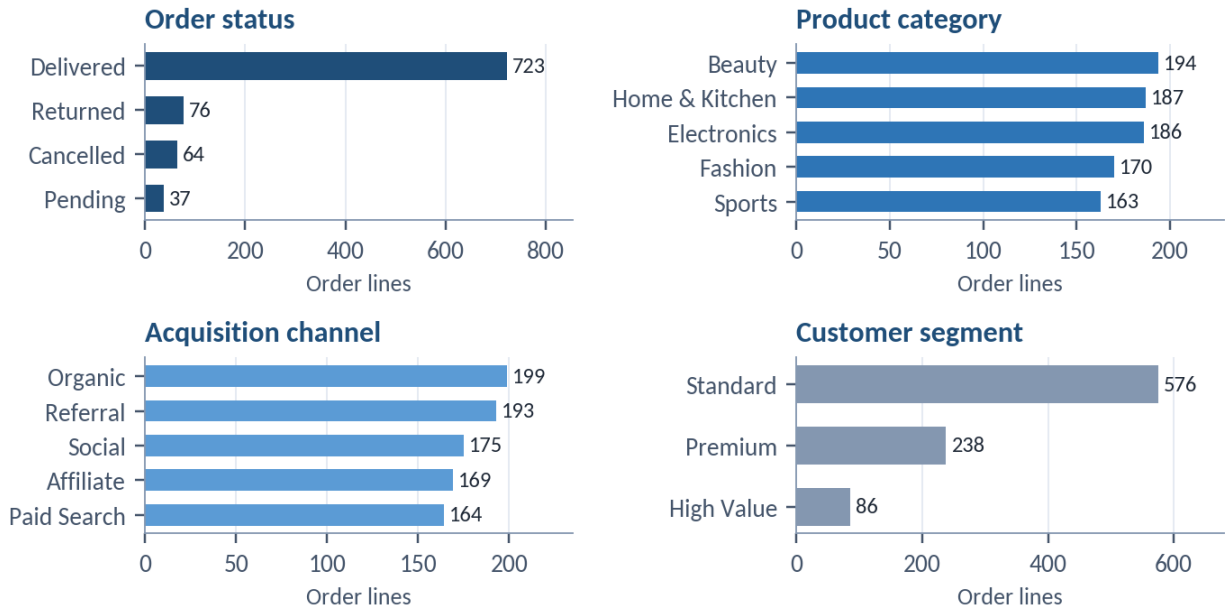


Figure 1 - Order lines by order status, product category, acquisition channel and customer segment (values from the distinct-count query; each panel adds up to 900).

Column	Value	Order lines
ORDER_STATUS	Delivered	723
	Returned	76
	Cancelled	64
	Pending	37
PRODUCT_CATEGORY	Beauty	194
	Home & Kitchen	187
	Electronics	186
	Fashion	170
	Sports	163

Column	Value	Order lines
ACQUISITION_CHANNEL	Organic	199
	Referral	193
	Social	175
	Affiliate	169
	Paid Search	164
CUSTOMER_SEGMENT	Standard	576
	Premium	238
	High Value	86

DUPLICATE, NULL AND REFUND CHECKS

```
-- *) DUPLICATE TRANSACTION_ID CHECK
SELECT TRANSACTION_ID, COUNT(*)
FROM marketplace_transactions
GROUP BY TRANSACTION_ID
HAVING COUNT(*) > 1;

-- *) NULL CHECKS
SELECT
  SUM(DISCOUNT_AMOUNT IS NULL) AS NULL_DISCOUNT,
  SUM(REFUND_AMOUNT IS NULL)   AS NULL_REFUND,
  SUM(PAYMENT_FEE IS NULL)     AS NULL_FEE,
  SUM(FULFILMENT_COST IS NULL) AS NULL_FULFIL
FROM marketplace_transactions;

-- *) STATUS WISE REFUND
SELECT ORDER_STATUS, COUNT(*) AS LINE_COUNT, SUM(REFUND_AMOUNT) AS TOTAL_REFUND
FROM marketplace_transactions
GROUP BY ORDER_STATUS;
```

Order status	Line count	Total refund
Delivered	723	0.00
Returned	76	37,945.29
Cancelled	64	0.00
Pending	37	0.00

REVIEW FLAGS F2, F11

The sample rows include order dates of 9/30/2025, 12/17/2025 and 12/20/2025, which are later than the reported newest date (9/9/2025). If order_timestamp is stored as text, MIN and MAX compare characters, not dates, so the true date range is not confirmed. Sample row T00002 also has an order date (2/12/2025) before its customer acquisition date (5/19/2025).

ANALYSIS QUESTION 01

Q1. Commercial performance bridge

From gross order value to contribution, with month-over-month growth checks.

Business question. Rebuild the commercial performance bridge. Calculate monthly GMV, delivered GMV, cancelled GMV, returned GMV, discounts, refunds, seller commission, payment fees, fulfilment costs and contribution proxy. Compare month-over-month growth in order count, GMV, delivered GMV and contribution proxy. Identify months with positive GMV growth but negative contribution growth.

Technique. `SUM(CASE…)` per order status, `COUNT(DISTINCT order_id)`, `LAG` for growth, `CASE` for the Yes/No flag.

Approach

- Check the GMV formula on single rows first (10-row test).
- Group by month and sum GMV by status: Delivered, Cancelled and Returned.
- Add discounts, refunds, payment fees and fulfilment costs, then compute the contribution proxy (Delivered rows only).
- Use LAG to compare each month with the one before, and flag months where GMV grows but contribution falls.

Q1 - STEP 1: GMV ON SINGLE ROWS

```
-- CALCULATES THE GMV OF ALL ROWS
SELECT
  TRANSACTION_ID,
  QUANTITY,
  UNIT_SELLING_PRICE,
  DISCOUNT_AMOUNT,
  (QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT AS GMV
FROM marketplace_transactions
LIMIT 10;
```

transaction_id	quantity	unit_selling_price	discount_amount	GMV
T00001	2	393.72	38.09	749.35
T00002	1	367.67	69.33	298.34
T00003	1	308.42	13.57	294.85
T00004	1	119.14	16.48	102.66
T00005	1	945.68	118.56	827.12
T00006	2	1,176.15	667.98	1,684.32
T00007	1	442.76	26.75	416.01
T00008	1	131.44	9.85	121.59
T00009	1	736.66	158.66	578.00
T00010	1	346.20	47.25	298.95

The source shows long decimals such as 298.34000000000003; values are rounded to 2 decimals here (F12).

Q1 - STEP 2: MONTHLY GMV

```
-- CALCULATE THE MONTHLY GMV
SELECT
  DATE_FORMAT(ORDER_TIMESTAMP, '%Y-%m-01') AS MONTH,
  COUNT(*) AS LINE_COUNT,
  SUM((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT) AS GMV
FROM marketplace_transactions
GROUP BY MONTH
ORDER BY MONTH DESC;
```

MONTH	LINE_COUNT	GMV
NULL	900	561,318.00 (561317.9999999997 as returned)

Q1 - STEP 3: FULL BRIDGE WITH GROWTH

```
-- REAL QUESTION ANSWER QUERY
WITH MONTHLY AS (
  SELECT
    DATE_FORMAT(ORDER_TIMESTAMP, '%Y-%m-01') AS MONTH,
    COUNT(*) AS LINE_COUNT,
    COUNT(DISTINCT ORDER_ID) AS ORDER_COUNT,
    ROUND(
      SUM(CASE WHEN ORDER_STATUS = 'Delivered'
        THEN ((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT) * SELLER_COMMISSION_RATE
        ELSE 0 END), 2) AS SELLER_COMMISSION,
    SUM((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT) AS GMV,
    SUM(CASE WHEN ORDER_STATUS = 'Delivered'
      THEN (QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT
      ELSE 0 END) AS DELIVERED_GMV,
    SUM(CASE WHEN ORDER_STATUS = 'Cancelled'
      THEN (QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT
      ELSE 0 END) AS CANCELLED_GMV,
    SUM(CASE WHEN ORDER_STATUS = 'Returned'
      THEN (QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT
      ELSE 0 END) AS RETURNED_GMV,
    SUM(CASE WHEN ORDER_STATUS = 'Delivered'
      THEN (QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT
           - REFUND_AMOUNT - PAYMENT_FEE - FULFILMENT_COST
      ELSE 0 END) AS CONTRIBUTION_PROXY,
    SUM(DISCOUNT_AMOUNT) AS DISCOUNTS,
    SUM(REFUND_AMOUNT) AS REFUNDS, SUM(PAYMENT_FEE) AS PAYMENT_FEES,
    SUM(FULFILMENT_COST) AS FULFILMENT_COSTS
  FROM marketplace_transactions
  GROUP BY MONTH
),
GROWTH AS (
  SELECT
    MONTH, DELIVERED_GMV, CANCELLED_GMV, RETURNED_GMV, DISCOUNTS, REFUNDS,
    SELLER_COMMISSION, PAYMENT_FEES, FULFILMENT_COSTS, CONTRIBUTION_PROXY,
    GMV, ORDER_COUNT,
    LAG(GMV) OVER (ORDER BY MONTH) AS PREV_GMV,
    ROUND(
      (GMV - LAG(GMV) OVER (ORDER BY MONTH))
      / LAG(GMV) OVER (ORDER BY MONTH) * 100, 2) AS GMV_GROWTH_PCT,
    LAG(ORDER_COUNT) OVER (ORDER BY MONTH) AS PREV_ORDER_COUNT,
    ROUND(
      (ORDER_COUNT - LAG(ORDER_COUNT) OVER (ORDER BY MONTH)) * 100.0
      / LAG(ORDER_COUNT) OVER (ORDER BY MONTH), 2) AS ORDER_GROWTH_PCT,
    LAG(DELIVERED_GMV) OVER (ORDER BY MONTH) AS PREV_DELIVERED_GMV,
    ROUND(
      (DELIVERED_GMV - LAG(DELIVERED_GMV) OVER (ORDER BY MONTH))
      / LAG(DELIVERED_GMV) OVER (ORDER BY MONTH) * 100, 2) AS DELIVERED_GMV_GROWTH_PCT,
    LAG(CONTRIBUTION_PROXY) OVER (ORDER BY MONTH) AS PREV_CONTRIBUTION_PROXY,
    ROUND(
      (CONTRIBUTION_PROXY - LAG(CONTRIBUTION_PROXY) OVER (ORDER BY MONTH))
      / LAG(CONTRIBUTION_PROXY) OVER (ORDER BY MONTH) * 100, 2) AS CONTRIBUTION_GROWTH_PCT
  FROM MONTHLY
)
SELECT
```

Q1 - STEP 3: FULL BRIDGE WITH GROWTH

```
  MONTH, ORDER_COUNT, GMV, DELIVERED_GMV, CANCELLED_GMV, RETURNED_GMV,
  DISCOUNTS, REFUNDS, SELLER_COMMISSION, PAYMENT_FEES, FULFILMENT_COSTS,
  CONTRIBUTION_PROXY, ORDER_GROWTH_PCT, GMV_GROWTH_PCT,
  DELIVERED_GMV_GROWTH_PCT, CONTRIBUTION_GROWTH_PCT,
  CASE WHEN GMV_GROWTH_PCT > 0 AND CONTRIBUTION_GROWTH_PCT < 0
       THEN 'Yes' ELSE 'No' END AS GMV_UP_CONTRIBUTION_DOWN
FROM GROWTH
ORDER BY MONTH;
```

Result

The query returned one row, because the month column is NULL for every line (F1). Values are listed as measure and value.

Measure	Value	Measure	Value
MONTH	NULL	DISCOUNTS	125,955.32
ORDER_COUNT	513	REFUNDS	37,945.29
GMV	561,318.00	SELLER_COMMISSION	55,751.85
DELIVERED_GMV	447,907.10	PAYMENT_FEES	16,497.70
CANCELLED_GMV	37,605.90	FULFILMENT_COSTS	54,689.80
RETURNED_GMV	49,621.57	CONTRIBUTION_PROXY	390,592.92
Growth columns (order, GMV, delivered GMV, contribution)	NULL	GMV_UP_CONTRIBUTION_DOWN	No

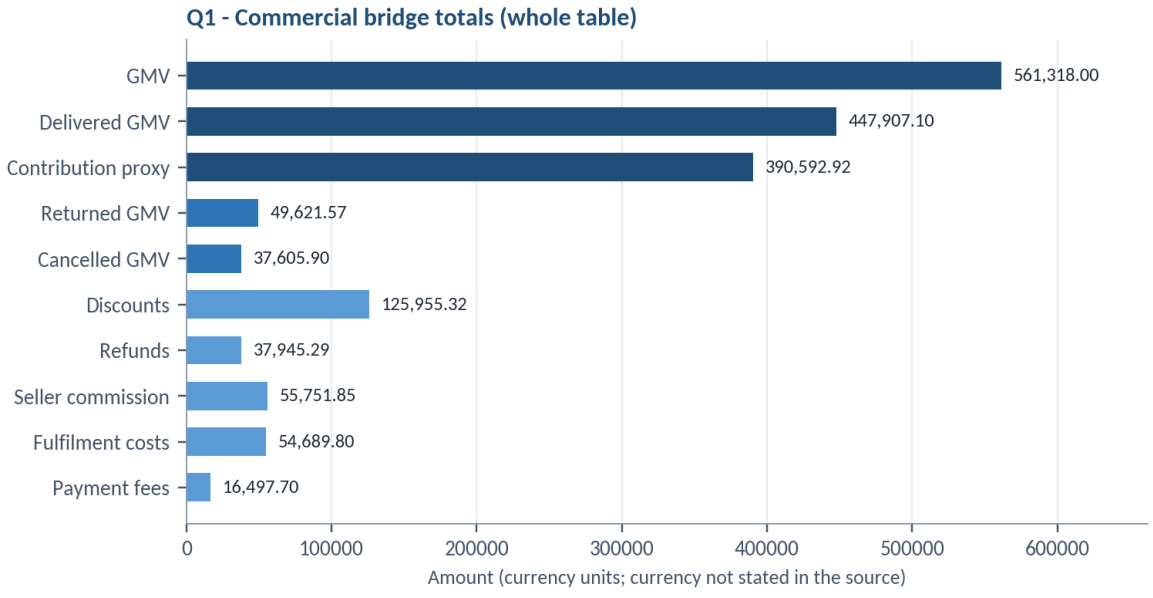


Figure 2 - Q1 totals for the whole table. Navy = GMV and contribution, blue = returned and cancelled value, light blue = costs and deductions. Values are the query results; no month split exists.

What the data shows

- Total GMV is 561,318.00 across 900 order lines and 513 distinct orders. Delivered GMV is 447,907.10, about 79.8% of GMV.
- Returned GMV is 49,621.57 (about 8.8% of GMV) and cancelled GMV is 37,605.90 (about 6.7%). The refunds of 37,945.29 all sit on Returned lines (Section 02).
- The contribution proxy is 390,592.92, about 87.2% of delivered GMV. Payment fees (16,497.70) and fulfilment costs (54,689.80) are totals over all rows, while the proxy only deducts them for Delivered rows (F3).
- No month-over-month answer is possible yet: growth values are NULL, so the 'No' in the last column carries no meaning.

> **REVIEW FLAGS F1, F3**
>
> The month column is NULL for all 900 lines. The sample rows show dates written like 8/14/2025, so check the stored type and format of order_timestamp before relying on any monthly result. The formula comments describe contribution and seller commission over all rows, but the queries apply them to Delivered rows only.

ANALYSIS QUESTION 02

Q2. Customer concentration and economic dependency

How much of delivered value and contribution comes from a few customers, and how many customers buy again.

Business question. Calculate the share of delivered GMV and contribution proxy generated by the top 1%, 5% and 10% of customers. Calculate repeat-purchase participation and identify customers whose delivered GMV is above the 90th percentile but whose contribution proxy is negative. Determine whether the largest customers are also the most profitable.

Technique. `NTILE(100)` to rank customers by delivered GMV, `PERCENT_RANK()` for the 90th percentile, `COUNT(DISTINCT order_id)` for repeat purchase.

Approach

- Total delivered GMV and contribution per customer (Delivered rows only).
- Cut customers into 100 buckets by delivered GMV; the top 1%, 5% and 10% are buckets 1, up to 5 and up to 10.
- Divide each band's totals by the grand totals to get shares.
- Count customers with 2 or more delivered orders for the repeat-purchase rate.
- List customers above the 90th GMV percentile with a negative contribution.

Q2 - CUSTOMER CONCENTRATION (TOP 1%, 5%, 10%)

```
-- FORMULA OF DELIVERED GMV - SUM((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT)
-- FORMULA OF CONTRIBUTION PROXY - SUM((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT
--   - REFUND_AMOUNT - PAYMENT_FEE - FULFILMENT_COST) AS CONTRIBUTION_PROXY.
-- CUSTOMER CONCENTRATION (TOP 1%, 5%, 10%)
WITH CUSTOMER_TOTALS AS (
  SELECT CUSTOMER_ID,
    SUM((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT) AS DELIVERED_GMV,
    SUM((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT
      - REFUND_AMOUNT - PAYMENT_FEE - FULFILMENT_COST) AS CONTRIBUTION_PROXY,
    COUNT(DISTINCT ORDER_ID) AS DELIVERED_ORDER_COUNT
  FROM marketplace_transactions
  WHERE ORDER_STATUS = 'Delivered'
  GROUP BY CUSTOMER_ID
),
RANKED AS (
  SELECT *,
    NTILE(100) OVER (ORDER BY DELIVERED_GMV DESC) AS PCT_BUCKET
  FROM CUSTOMER_TOTALS
)
SELECT 'Top 1%' AS CONCENTRATION_BAND,
  COUNT(*) AS CUSTOMER_COUNT,
  SUM(DELIVERED_GMV) AS DELIVERED_GMV,
  SUM(CONTRIBUTION_PROXY) AS CONTRIBUTION_PROXY,
  ROUND(100 * SUM(DELIVERED_GMV)
    / (SELECT SUM(DELIVERED_GMV) FROM CUSTOMER_TOTALS), 2) AS SHARE_OF_TOTAL_GMV_PCT,
  ROUND(100 * SUM(CONTRIBUTION_PROXY)
    / (SELECT SUM(CONTRIBUTION_PROXY) FROM CUSTOMER_TOTALS), 2) AS SHARE_OF_TOTAL_CONTRIBUTION_PCT
FROM RANKED
WHERE PCT_BUCKET <= 1
UNION ALL
SELECT 'Top 5%' AS CONCENTRATION_BAND,
  COUNT(*) AS CUSTOMER_COUNT,
  SUM(DELIVERED_GMV) AS DELIVERED_GMV,
  SUM(CONTRIBUTION_PROXY) AS CONTRIBUTION_PROXY,
  ROUND(100 * SUM(DELIVERED_GMV)
    / (SELECT SUM(DELIVERED_GMV) FROM CUSTOMER_TOTALS), 2) AS SHARE_OF_TOTAL_GMV_PCT,
  ROUND(100 * SUM(CONTRIBUTION_PROXY)
    / (SELECT SUM(CONTRIBUTION_PROXY) FROM CUSTOMER_TOTALS), 2) AS SHARE_OF_TOTAL_CONTRIBUTION_PCT
FROM RANKED
WHERE PCT_BUCKET <= 5
UNION ALL
SELECT 'Top 10%' AS CONCENTRATION_BAND,
```

Q2 - CUSTOMER CONCENTRATION (TOP 1%, 5%, 10%)

```
  COUNT(*) AS CUSTOMER_COUNT,
  SUM(DELIVERED_GMV) AS DELIVERED_GMV,
  SUM(CONTRIBUTION_PROXY) AS CONTRIBUTION_PROXY,
  ROUND(100 * SUM(DELIVERED_GMV)
    / (SELECT SUM(DELIVERED_GMV) FROM CUSTOMER_TOTALS), 2) AS SHARE_OF_TOTAL_GMV_PCT,
  ROUND(100 * SUM(CONTRIBUTION_PROXY)
    / (SELECT SUM(CONTRIBUTION_PROXY) FROM CUSTOMER_TOTALS), 2) AS SHARE_OF_TOTAL_CONTRIBUTION_PCT
FROM RANKED
WHERE PCT_BUCKET <= 10;
```

Result - concentration

Band	Customers	Delivered GMV	Contribution proxy	% of total GMV	% of total contribution
Top 1%	3	22,706.82	21,122.47	5.07	5.41
Top 5%	15	90,022.69	82,584.49	20.10	21.14
Top 10%	30	152,736.03	139,119.76	34.10	35.62

Q2 - REPEAT-PURCHASE PARTICIPATION

```
-- REPEAT-PURCHASE PARTICIPATION
WITH CUSTOMER_ORDERS AS (
  SELECT CUSTOMER_ID,
    COUNT(DISTINCT ORDER_ID) AS DELIVERED_ORDER_COUNT
  FROM marketplace_transactions
  WHERE ORDER_STATUS = 'Delivered'
  GROUP BY CUSTOMER_ID
)
SELECT COUNT(*) AS CUSTOMERS_WITH_1_PLUS_ORDERS,
  SUM(CASE WHEN DELIVERED_ORDER_COUNT >= 2 THEN 1 ELSE 0 END) AS CUSTOMERS_WITH_2_PLUS_ORDERS,
  ROUND(100.0 * SUM(CASE WHEN DELIVERED_ORDER_COUNT >= 2 THEN 1 ELSE 0 END)
    / NULLIF(COUNT(*), 0), 2) AS REPEAT_PURCHASE_RATE_PCT
FROM CUSTOMER_ORDERS;
```

Result - repeat purchase

Customers with 1+ delivered orders	Customers with 2+ delivered orders	Repeat purchase rate %
226	191	84.51

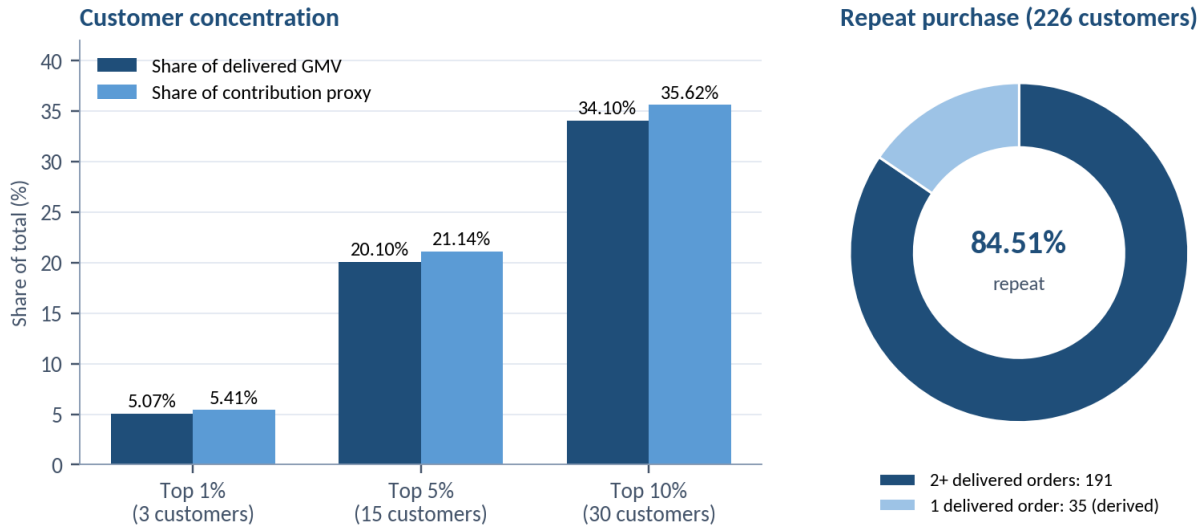


Figure 3 - Left: share of delivered GMV and of contribution proxy held by the top customer bands. Right: customers with 2+ delivered orders; the '1 order' count (35) is derived as 226 - 191.

Q2 - LARGE CUSTOMERS WITH NEGATIVE CONTRIBUTION

```
-- BIG CUSTOMERS (ABOVE 90TH PERCENTILE) WITH NEGATIVE CONTRIBUTION
WITH CUSTOMER_TOTALS AS (
  SELECT CUSTOMER_ID,
    SUM((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT) AS DELIVERED_GMV,
    SUM((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT
      - REFUND_AMOUNT - PAYMENT_FEE - FULFILMENT_COST) AS CONTRIBUTION_PROXY,
    COUNT(DISTINCT ORDER_ID) AS DELIVERED_ORDER_COUNT
  FROM marketplace_transactions
  WHERE ORDER_STATUS = 'Delivered'
  GROUP BY CUSTOMER_ID
),
PCT AS (
  SELECT *,
    PERCENT_RANK() OVER (ORDER BY DELIVERED_GMV) AS GMV_PERCENTILE
  FROM CUSTOMER_TOTALS
)
SELECT CUSTOMER_ID,
  DELIVERED_GMV,
  CONTRIBUTION_PROXY,
  DELIVERED_ORDER_COUNT,
  ROUND(GMV_PERCENTILE, 2) AS GMV_PERCENTILE,
  CASE WHEN CONTRIBUTION_PROXY < 0 THEN 'Yes' ELSE 'No' END AS NEGATIVE_CONTRIBUTION
FROM PCT
WHERE GMV_PERCENTILE > 0.90
  AND CONTRIBUTION_PROXY < 0
ORDER BY DELIVERED_GMV DESC;
```

No result table for this query is included in the source (F4).

What the data shows

- The top 10% of customers (30 customers) account for 34.10% of delivered GMV and 35.62% of contribution proxy. The top 5% hold 20.10% and 21.14%; the top 1% hold 5.07% and 5.41%.
- In each band the contribution share is a little higher than the GMV share (by 0.34, 1.04 and 1.52 points). In these results the largest customers are not diluting profit; the gap is small.
- 191 of 226 customers with a delivered order (84.51%) placed 2 or more delivered orders.
- Whether any large customer has a negative contribution cannot be read from the source, because that result is missing.

REVIEW FLAGS F4, F5, F6

The source repeats the concentration table under the repeat-purchase query; the repeat-purchase result (226, 191, 84.51) appears after the next query and is placed here with its own query. The repeat rate in Q4 is 81.28% on 235 customers, because the base differs (F5). Order IDs may be shared by several customers (F6).

ANALYSIS QUESTION 03

Q3. Discount dependency and commercial quality

Where heavy discounts go together with weak economics, by category, channel, customer segment and single transaction.

Business question. Calculate discount rates by product category and acquisition channel. Identify categories where higher discount rates coincide with lower contribution per delivered order. Compare customer segments by average order value, repeat-purchase frequency, discount dependency, refund burden and contribution proxy. Identify high-value transactions with weak post-discount economics.

Approach

- Discount rate = discounts ÷ list value (quantity × unit_list_price), by category and by channel.
- Score each category: 'High' risk when its discount rate is above the average and contribution per order is below the average; 'Low' when the reverse; otherwise 'Medium'.
- Compare the three customer segments on order value, orders per customer, discount dependency, refund burden and contribution.
- Find Delivered or Returned lines in the top 10% of line GMV whose contribution is below half of the line GMV.

Q3 - DISCOUNT RATE BY PRODUCT CATEGORY

```
-- DISCOUNT RATE BY PRODUCT CATEGORY
SELECT PRODUCT_CATEGORY,
  ROUND(100 * SUM(DISCOUNT_AMOUNT)
    / SUM(QUANTITY * UNIT_LIST_PRICE), 2) AS DISCOUNT_RATE_PCT
FROM marketplace_transactions
GROUP BY PRODUCT_CATEGORY
ORDER BY DISCOUNT_RATE_PCT DESC;
```

Q3 - DISCOUNT RATE BY ACQUISITION CHANNEL

```
-- DISCOUNT RATE BY ACQUISITION CHANNEL
SELECT ACQUISITION_CHANNEL,
  ROUND(100 * SUM(DISCOUNT_AMOUNT)
    / SUM(QUANTITY * UNIT_LIST_PRICE), 2) AS DISCOUNT_RATE_PCT
FROM marketplace_transactions
GROUP BY ACQUISITION_CHANNEL
ORDER BY DISCOUNT_RATE_PCT DESC;
```

Product category	Discount rate %	Acquisition channel	Discount rate %
Beauty	16.53	Affiliate	17.06
Fashion	16.46	Referral	15.91
Home & Kitchen	16.39	Organic	15.35
Electronics	14.49	Paid Search	14.97
Sports	13.22	Social	14.04

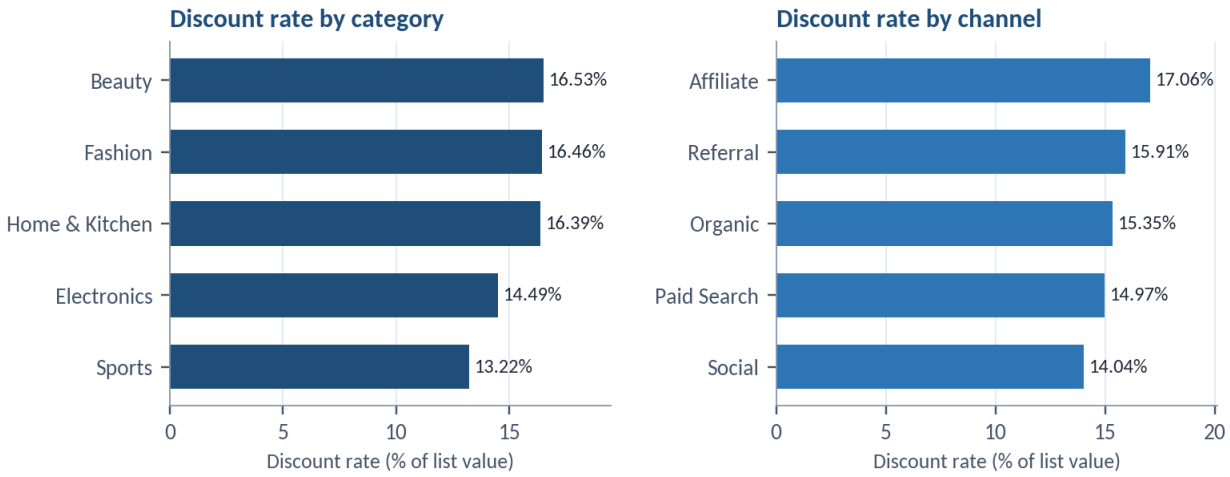


Figure 4 - Discount rate (discount ÷ list value) by category and by acquisition channel.

Q3 - CATEGORY COMMERCIAL QUALITY

```
-- CATEGORY COMMERCIAL QUALITY
WITH CAT AS (
  SELECT PRODUCT_CATEGORY,
    100 * SUM(DISCOUNT_AMOUNT)
      / NULLIF(SUM(QUANTITY * UNIT_LIST_PRICE), 0) AS DISCOUNT_RATE_PCT,
    100 * SUM(REFUND_AMOUNT)
      / NULLIF(SUM((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT), 0) AS REFUND_RATE_PCT,
    COUNT(DISTINCT CASE WHEN ORDER_STATUS = 'Delivered' THEN ORDER_ID END) AS DELIVERED_ORDERS,
    SUM(CASE WHEN ORDER_STATUS = 'Delivered'
      THEN (QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT
           - REFUND_AMOUNT - PAYMENT_FEE - FULFILMENT_COST
      ELSE 0 END) AS DELIVERED_CONTRIBUTION
  FROM marketplace_transactions
  GROUP BY PRODUCT_CATEGORY
),
SCORED AS (
  SELECT *,
    DELIVERED_CONTRIBUTION / NULLIF(DELIVERED_ORDERS, 0) AS CONTRIBUTION_PER_ORDER
  FROM CAT
)
SELECT PRODUCT_CATEGORY,
  ROUND(DISCOUNT_RATE_PCT, 2) AS DISCOUNT_RATE_PCT,
  ROUND(CONTRIBUTION_PER_ORDER, 2) AS CONTRIBUTION_PER_ORDER,
  ROUND(REFUND_RATE_PCT, 2) AS REFUND_RATE_PCT,
  CASE WHEN DISCOUNT_RATE_PCT > AVG(DISCOUNT_RATE_PCT) OVER ()
       AND CONTRIBUTION_PER_ORDER < AVG(CONTRIBUTION_PER_ORDER) OVER () THEN 'High'
     WHEN DISCOUNT_RATE_PCT <= AVG(DISCOUNT_RATE_PCT) OVER ()
       AND CONTRIBUTION_PER_ORDER >= AVG(CONTRIBUTION_PER_ORDER) OVER () THEN 'Low'
     ELSE 'Medium' END AS COMMERCIAL_RISK,
  ROUND(0.10 * DELIVERED_CONTRIBUTION, 2) AS CONTRIBUTION_EXPOSED_TO_10PCT_GMV_DROP,
  RANK() OVER (ORDER BY DELIVERED_CONTRIBUTION DESC) AS EXPOSURE_RANK
FROM SCORED
ORDER BY EXPOSURE_RANK;
```

Result - category quality

Category	Discount rate %	Contribution per order	Refund rate %	Commercial risk	Contribution exposed to 10% GMV drop	Exposure rank
Electronics	14.49	721.55	5.36	Low	9,596.65	1
Beauty	16.53	618.88	5.64	Medium	8,973.77	2
Home & Kitchen	16.39	603.34	6.78	Medium	7,964.03	3
Fashion	16.46	504.34	9.11	High	6,354.63	4
Sports	13.22	546.04	7.94	Medium	6,170.20	5

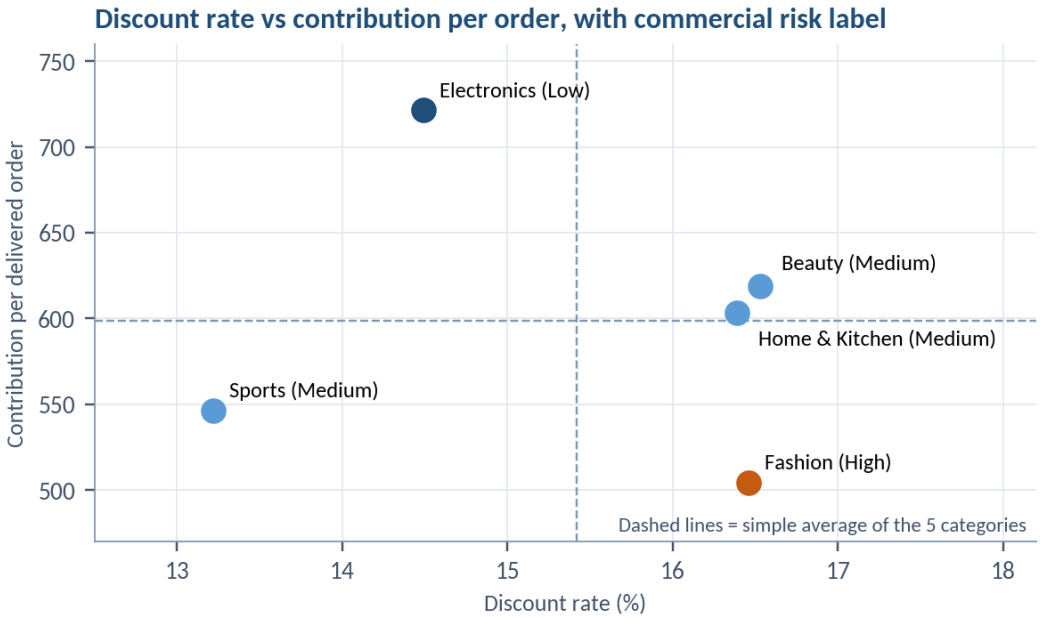


Figure 5 - Discount rate against contribution per delivered order. Colour shows the query's risk label: navy = Low, light blue = Medium, orange = High.

Q3 - CUSTOMER SEGMENT COMPARISON

```
-- CUSTOMER SEGMENT COMPARISON
SELECT
  CUSTOMER_SEGMENT,
  COUNT(DISTINCT CASE WHEN ORDER_STATUS = 'Delivered' THEN CUSTOMER_ID END) AS DELIVERED_CUSTOMERS,
  COUNT(DISTINCT CASE WHEN ORDER_STATUS = 'Delivered' THEN ORDER_ID END) AS DELIVERED_ORDERS,
  ROUND(SUM(CASE WHEN ORDER_STATUS = 'Delivered'
      THEN (QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT ELSE 0 END)
    / NULLIF(COUNT(DISTINCT CASE WHEN ORDER_STATUS = 'Delivered' THEN ORDER_ID END), 0), 2)
    AS AVG_ORDER_VALUE,
  ROUND(COUNT(DISTINCT CASE WHEN ORDER_STATUS = 'Delivered' THEN ORDER_ID END)
    / NULLIF(COUNT(DISTINCT CASE WHEN ORDER_STATUS = 'Delivered' THEN CUSTOMER_ID END), 0), 2)
    AS ORDERS_PER_CUSTOMER,
  ROUND(100 * SUM(DISCOUNT_AMOUNT)
    / NULLIF(SUM(QUANTITY * UNIT_LIST_PRICE), 0), 2) AS DISCOUNT_DEPENDENCY_PCT,
  ROUND(100 * SUM(REFUND_AMOUNT)
    / NULLIF(SUM((QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT), 0), 2) AS REFUND_BURDEN_PCT,
  ROUND(SUM(CASE WHEN ORDER_STATUS = 'Delivered'
      THEN (QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT
           - REFUND_AMOUNT - PAYMENT_FEE - FULFILMENT_COST
      ELSE 0 END), 2) AS CONTRIBUTION_PROXY
FROM marketplace_transactions
GROUP BY CUSTOMER_SEGMENT
ORDER BY CONTRIBUTION_PROXY DESC;
```

Result - customer segments

Segment	Delivered customers	Delivered orders	Avg order value	Orders per customer	Discount dependency %	Refund burden %	Contribution proxy
Standard	204	333	894.33	1.63	15.28	8.35	261,402.66
High Value	60	67	531.40	1.12	15.71	6.43	30,344.33
Premium	130	175	654.24	1.35	15.96	2.65	98,845.93

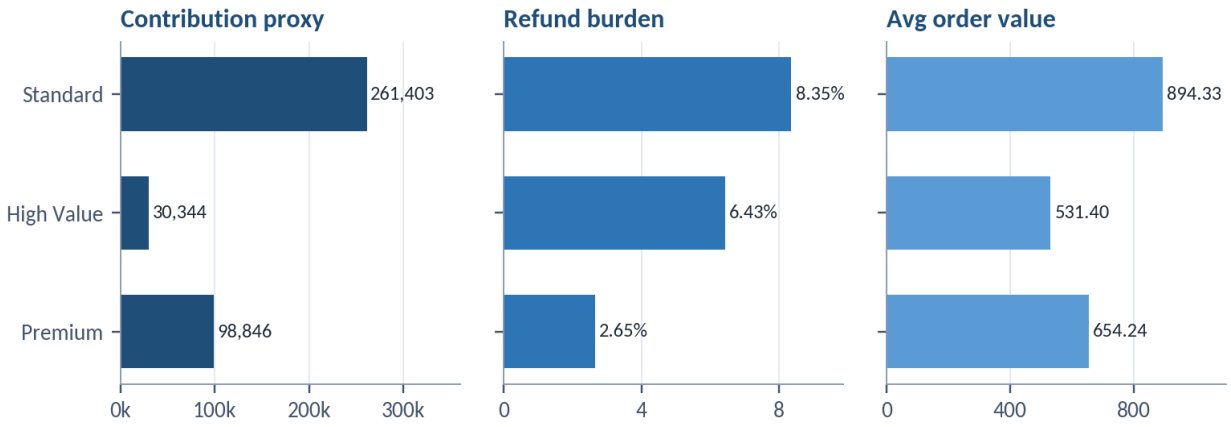


Figure 6 - Segment comparison: contribution proxy, refund burden and average order value.

Q3 - HIGH-VALUE TRANSACTIONS WITH WEAK ECONOMICS

```
-- HIGH-VALUE TRANSACTIONS WITH WEAK POST-DISCOUNT ECONOMICS
WITH LINES_COUNT AS (
  SELECT TRANSACTION_ID, ORDER_STATUS, PRODUCT_CATEGORY,
    (QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT AS LINE_GMV,
    100 * DISCOUNT_AMOUNT / NULLIF(QUANTITY * UNIT_LIST_PRICE, 0) AS DISCOUNT_RATE_PCT,
    REFUND_AMOUNT,
    (QUANTITY * UNIT_SELLING_PRICE) - DISCOUNT_AMOUNT
      - REFUND_AMOUNT - PAYMENT_FEE - FULFILMENT_COST AS LINE_CONTRIBUTION
  FROM marketplace_transactions
  WHERE ORDER_STATUS IN ('Delivered', 'Returned')
),
PCT AS (
  SELECT *,
    PERCENT_RANK() OVER (ORDER BY LINE_GMV) AS GMV_PERCENTILE
  FROM LINES_COUNT
)
SELECT TRANSACTION_ID, ORDER_STATUS, PRODUCT_CATEGORY,
  ROUND(LINE_GMV, 2) AS LINE_GMV,
  ROUND(DISCOUNT_RATE_PCT, 2) AS DISCOUNT_RATE_PCT,
  REFUND_AMOUNT,
  ROUND(LINE_CONTRIBUTION, 2) AS LINE_CONTRIBUTION,
  ROUND(100 * LINE_CONTRIBUTION / NULLIF(LINE_GMV, 0), 2) AS CONTRIBUTION_MARGIN_PCT
FROM PCT
WHERE GMV_PERCENTILE > 0.90
  AND LINE_CONTRIBUTION < 0.5 * LINE_GMV
ORDER BY CONTRIBUTION_MARGIN_PCT;
```

Result - high-value lines with weak post-discount economics

Transaction	Status	Category	Line GMV	Discount rate %	Refund amount	Line contribution	Margin %
T00541	Returned	Fashion	1,768.46	32.65	2,748.96	-1,102.83	-62.36
T00343	Returned	Sports	2,004.50	8.84	2,106.61	-215.47	-10.75
T00433	Returned	Beauty	1,789.07	6.57	1,811.03	-152.80	-8.54
T00235	Returned	Fashion	1,397.52*	8.35	1,436.71	-99.99	-7.15
T00755	Returned	Fashion	1,725.96	8.81	1,709.89	-24.49	-1.42
T00362	Returned	Sports	1,686.52	3.36	1,190.98	341.27	20.24
T00104	Returned	Electronics	2,232.19	6.45	1,211.81	877.14	39.30
T00498	Returned	Home & Kitchen	3,242.20	1.38	1,776.00	1,345.38	41.50
T00672	Returned	Home & Kitchen	1,997.89	3.66	1,072.49	845.22	42.31
T00447	Returned	Beauty	2,142.39	8.57	1,087.70	907.77	42.37

** Partly hidden by a tooltip in the source screenshot; read as 1,397.52 and checked against its margin (-99.99 ÷ 1,397.52 = -7.15%). Please confirm (F13).*

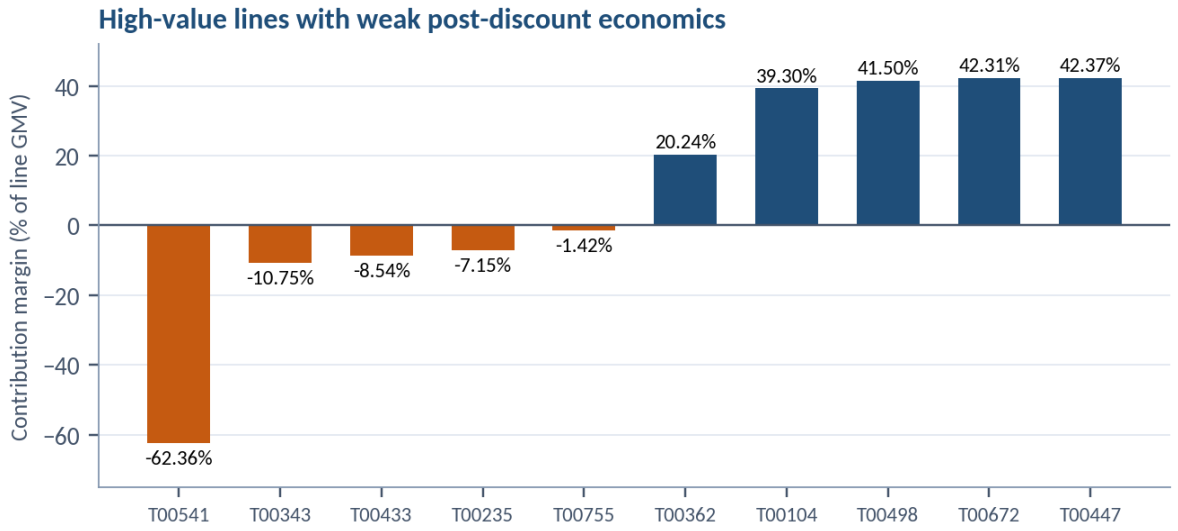


Figure 7 - Contribution margin of the 10 high-value lines. Orange = negative margin.

What the data shows

- Beauty (16.53%), Fashion (16.46%) and Home & Kitchen (16.39%) have the highest discount rates; Sports has the lowest (13.22%). By channel, Affiliate is highest (17.06%) and Social lowest (14.04%).
- Fashion is the only category labelled High risk: above-average discount rate, the lowest contribution per order (504.34) and the highest refund rate (9.11%). Electronics is labelled Low, with the highest contribution per order (721.55).
- Discount rate alone does not rank contribution per order: Sports has the lowest discount rate but only the second-lowest contribution per order.
- Standard customers give 261,402.66, about 66.9% of the 390,592.92 contribution proxy, and carry the highest refund burden (8.35%). Premium has the lowest refund burden (2.65%). The 'High Value' segment has the lowest average order value (531.40).
- All 10 high-value weak-economics lines are Returned. Five have a negative contribution (T00541, T00343, T00433, T00235, T00755). Four of these five have a refund larger than the line GMV (all except T00755) (F8).

REVIEW FLAGS F7, F8, F13, F15

Segment columns mix scopes: customers, orders and contribution use Delivered rows, while discount dependency and refund burden use all rows (F15). Distinct delivered customers add to 394 across segments, but only 226 customers have delivered orders, so a customer seems to appear in more than one segment (F7). Refunds above line GMV need a check against the refund rules (F8).

ANALYSIS QUESTION 04

Q4. Acquisition cohort quality

Cohorts by customer acquisition month: contribution, repeat purchase and early economics.

Business question. Build acquisition-month cohorts. Compare first-purchase contribution, 90-day cumulative contribution and repeat-purchase rate by acquisition channel. Identify cohorts where first-purchase economics are positive but 90-day economics deteriorate, and flag incomplete 90-day windows.

Approach in the submitted queries

- Turn customer_acquisition_date into a year-month with STR_TO_DATE and DATE_FORMAT.
- Count customers and delivered orders per cohort, and add delivered GMV and contribution proxy.
- Compute contribution per delivered order, and the repeat-customer rate (2+ delivered orders) per cohort.
- The queries group by acquisition month. First-purchase contribution, 90-day windows and channel splits are not part of them (F9).

Q4 - COHORT TOTALS

```
-- Acquisition Cohort Analysis
SELECT
  DATE_FORMAT(
    STR_TO_DATE(customer_acquisition_date, '%Y-%m-%d'),
    '%Y-%m') AS acquisition_month,
  COUNT(DISTINCT customer_id) AS total_customers,
  COUNT(DISTINCT CASE
    WHEN order_status = 'Delivered' THEN order_id END) AS delivered_orders,
  ROUND(SUM(
    CASE WHEN order_status = 'Delivered'
    THEN quantity * unit_selling_price - discount_amount
    ELSE 0 END), 2) AS delivered_gmv,
  ROUND(SUM(
    CASE WHEN order_status = 'Delivered'
    THEN (quantity * unit_selling_price - discount_amount)
         - refund_amount - payment_fee - fulfilment_cost
    ELSE 0 END), 2) AS contribution_proxy
FROM marketplace_transactions
GROUP BY acquisition_month
ORDER BY acquisition_month;
```

Q4 - CONTRIBUTION PER DELIVERED ORDER

```
-- Acquisition Cohort Contribution per Delivered Order
SELECT
  DATE_FORMAT(
    STR_TO_DATE(customer_acquisition_date, '%Y-%m-%d'),
    '%Y-%m') AS acquisition_month,
  COUNT(DISTINCT customer_id) AS total_customers,
  COUNT(DISTINCT CASE
    WHEN order_status = 'Delivered' THEN order_id END) AS delivered_orders,
  ROUND(
    SUM(
      CASE WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount)
           - refund_amount - payment_fee - fulfilment_cost ELSE 0 END)
    / NULLIF(
      COUNT(DISTINCT CASE
        WHEN order_status = 'Delivered' THEN order_id
      END), 0), 2) AS contribution_per_delivered_order
FROM marketplace_transactions
GROUP BY acquisition_month
ORDER BY contribution_per_delivered_order DESC;
```

Q4 - REPEAT CUSTOMER RATE BY COHORT

```
-- Which customer acquisition cohorts have the highest repeat customer rate, and how does
-- repeat purchasing behavior vary across acquisition months?
SELECT
  acquisition_month,
  COUNT(*) AS total_customers,
  SUM(CASE
    WHEN delivered_orders >= 2 THEN 1 ELSE 0 END) AS repeat_customers,
  ROUND(
    SUM(CASE
      WHEN delivered_orders >= 2 THEN 1 ELSE 0 END) * 100.0
    / COUNT(*), 2) AS repeat_customer_rate_pct
FROM (
  SELECT
    customer_id,
    DATE_FORMAT(
      STR_TO_DATE(
        MIN(customer_acquisition_date), '%Y-%m-%d'), '%Y-%m') AS acquisition_month,
    COUNT(DISTINCT CASE
      WHEN order_status = 'Delivered' THEN order_id END) AS delivered_orders
  FROM marketplace_transactions
  GROUP BY customer_id
) AS customer_cohorts
GROUP BY acquisition_month
ORDER BY acquisition_month;
```

Q4 - TOTAL CONTRIBUTION AND CONTRIBUTION PER ORDER

```
-- Which acquisition cohorts generate the highest total contribution and contribution per
-- delivered order, and how can the business use these insights to improve customer
-- acquisition strategy?
SELECT
  DATE_FORMAT(
    STR_TO_DATE(customer_acquisition_date, '%Y-%m-%d'), '%Y-%m') AS acquisition_month,
  COUNT(DISTINCT customer_id) AS total_customers,
  COUNT(DISTINCT CASE
    WHEN order_status = 'Delivered'
    THEN order_id
  END) AS delivered_orders,
  ROUND(
    SUM(CASE
      WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount) - refund_amount -
           payment_fee - fulfilment_cost ELSE 0 END), 2) AS total_contribution,
  ROUND(
    SUM(CASE
      WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount) - refund_amount - payment_fee
           - fulfilment_cost ELSE 0 END) /
    NULLIF(COUNT(DISTINCT CASE
      WHEN order_status = 'Delivered' THEN order_id END), 0), 2) AS contribution_per_order
FROM marketplace_transactions
GROUP BY acquisition_month
ORDER BY total_contribution DESC;
```

Q4 - REPEAT RATE AND CONTRIBUTION TOGETHER

```
-- What recommendations can the business make to improve customer acquisition quality,
-- repeat purchases, and contribution from each acquisition cohort?
SELECT
  acquisition_month,
  COUNT(*) AS total_customers,
  SUM(CASE WHEN delivered_orders >= 2 THEN 1 ELSE 0 END) AS repeat_customers,
  ROUND(
    100.0 * SUM(CASE
      WHEN delivered_orders >= 2 THEN 1 ELSE 0 END) / COUNT(*), 2) AS repeat_customer_rate_pct,
  SUM(delivered_orders) AS delivered_orders,
  ROUND(SUM(total_contribution), 2) AS total_contribution,
  ROUND(
    SUM(total_contribution)
    / NULLIF(SUM(delivered_orders), 0), 2) AS contribution_per_order
FROM (
  SELECT
    customer_id,
    DATE_FORMAT(
```

Q4 - REPEAT RATE AND CONTRIBUTION TOGETHER

```
      STR_TO_DATE(
        MIN(customer_acquisition_date), '%Y-%m-%d'), '%Y-%m') AS acquisition_month,
    COUNT(DISTINCT CASE
      WHEN order_status = 'Delivered' THEN order_id END) AS delivered_orders,
    SUM(CASE
      WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount)
           - refund_amount - payment_fee - fulfilment_cost ELSE 0 END) AS total_contribution
  FROM marketplace_transactions
  GROUP BY customer_id
) AS customer_cohorts
GROUP BY acquisition_month
ORDER BY total_contribution DESC;
```

Results (every query returned one row; acquisition_month = NULL)

Query	Customers	Delivered orders	Delivered GMV	Contribution	Contrib. per order	Repeat customers	Repeat rate %
Cohort totals	235	447	447,907.10	390,592.92	-	-	-
Contribution per delivered order	235	447	-	-	873.81	-	-
Repeat customer rate	235	-	-	-	-	191	81.28
Total contribution and per order	235	447	-	390,592.92	873.81	-	-
Repeat rate and contribution together	235	721	-	390,592.92	541.74	191	81.28

235	191	81.28%	390,592.92
CUSTOMERS	REPEAT CUSTOMERS	REPEAT RATE	CONTRIBUTION PROXY

What the data shows

- Across all 235 customers, 191 (81.28%) have 2 or more delivered orders, and the total contribution proxy is 390,592.92. This total matches Q1.
- No cohort comparison is possible: because acquisition_month is NULL, all customers fall into one group (F1).
- Contribution per order differs by how orders are counted: 873.81 on 447 distinct order IDs, and 541.74 on 721 customer-order pairs (F6).

REVIEW FLAGS F1, F5, F6, F9

The sample rows show acquisition dates like 7/6/2025, while the queries read '%Y-%m-%d', which probably explains the NULL month. Items asked in the question but not found in the submitted queries: first-purchase contribution, 90-day cumulative contribution, repeat rate by acquisition channel, cohorts with positive first purchase but weaker 90-day economics, and flags for incomplete 90-day windows.

ANALYSIS QUESTION 05

Q5. Commercial downside scenario

What a 10% cut in discount spend saves, and how much delivered volume could fall before the saving is cancelled.

Business question. Quantify current monthly contribution by category. Calculate the break-even delivered-GMV decline that eliminates the benefit of a 10% reduction in discount expense. Identify categories with the smallest tolerance for volume decline.

Approach

- Discount saving = 10% of discount_amount on Delivered rows.
- Contribution at risk = current delivered contribution of the category.
- Break-even decline % = discount saving ÷ current contribution × 100, as written in the queries (F10).
- A lower percentage means the category can lose less volume before the saving is cancelled.

Q5 - BY MONTH AND CATEGORY (MONTH RETURNS NULL)

```
-- Current contribution and discount expense by category
SELECT
  DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
  product_category,
  ROUND(
    SUM(CASE
      WHEN order_status = 'Delivered' THEN discount_amount ELSE 0
    END) * 0.10, 2) AS discount_saving,
  ROUND(
    SUM(CASE
      WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount)
           - refund_amount - payment_fee - fulfilment_cost
      ELSE 0 END), 2) AS contribution_at_risk,
  ROUND(
    (SUM(CASE
        WHEN order_status = 'Delivered'
        THEN discount_amount ELSE 0 END) * 0.10)
    / NULLIF(SUM(CASE
        WHEN order_status = 'Delivered'
        THEN (quantity * unit_selling_price - discount_amount)
             - refund_amount - payment_fee - fulfilment_cost
        ELSE 0 END), 0) * 100, 2) AS break_even_gmv_decline_pct
FROM marketplace_transactions
GROUP BY
  DATE_FORMAT(order_timestamp, '%Y-%m'),
  product_category
ORDER BY
  month,
  break_even_gmv_decline_pct ASC;
```

Q5 - 10% DISCOUNT REDUCTION SCENARIO

```
-- 10% Discount Reduction Scenario
SELECT
  product_category,
  ROUND(SUM(CASE
    WHEN order_status = 'Delivered'
    THEN discount_amount
    ELSE 0 END) * 0.10, 2) AS discount_saving,
  ROUND(SUM(CASE
    WHEN order_status = 'Delivered'
    THEN (quantity * unit_selling_price - discount_amount)
         - refund_amount - payment_fee - fulfilment_cost
    ELSE 0 END), 2) AS contribution_at_risk,
  ROUND(
    100.0 * (SUM(CASE
      WHEN order_status = 'Delivered' THEN discount_amount ELSE 0 END) * 0.10)
    / NULLIF(SUM(CASE WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount)
           - refund_amount - payment_fee - fulfilment_cost
      ELSE 0 END), 0), 2) AS break_even_gmv_decline_pct
FROM marketplace_transactions
GROUP BY product_category
ORDER BY break_even_gmv_decline_pct ASC;
```

Result - break-even by category

Category	Discount saving	Contribution at risk	Break-even GMV decline %
Sports	1,288.46	61,702.04	2.09
Electronics	2,139.85	95,966.49	2.23
Fashion	1,796.34	63,546.35	2.83
Home & Kitchen	2,264.34	79,640.35	2.84
Beauty	2,705.58	89,737.69	3.01

Q5 - CURRENT CONTRIBUTION BY CATEGORY

```
-- Monthly contribution by category
SELECT
  DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
  product_category,
  COUNT(DISTINCT CASE
    WHEN order_status = 'Delivered'
    THEN order_id
  END) AS delivered_orders,
  ROUND(SUM(CASE
    WHEN order_status = 'Delivered'
    THEN quantity * unit_selling_price - discount_amount
    ELSE 0
  END), 2) AS delivered_gmv,
  ROUND(SUM(CASE
    WHEN order_status = 'Delivered'
    THEN discount_amount
    ELSE 0
  END), 2) AS discount_expense,
  ROUND(SUM(CASE
    WHEN order_status = 'Delivered'
    THEN (quantity * unit_selling_price - discount_amount)
         - refund_amount - payment_fee - fulfilment_cost
    ELSE 0
  END), 2) AS current_contribution
FROM marketplace_transactions
GROUP BY
  DATE_FORMAT(order_timestamp, '%Y-%m'),
  product_category
ORDER BY
  month,
  product_category;
```

Result - current contribution by category

Category	Delivered orders	Delivered GMV	Discount expense	Current contribution
Beauty	145	101,829.78	27,055.84	89,737.69
Electronics	133	108,362.46	21,398.49	95,966.49
Fashion	126	74,792.31	17,963.44	63,546.35
Home & Kitchen	132	91,523.91	22,643.42	79,640.35
Sports	113	71,398.64	12,884.62	61,702.04

In the source the month column of both results is NULL, so these are category totals for the whole table (F1). The five contributions add up to 390,592.92, matching Q1 and Q2.

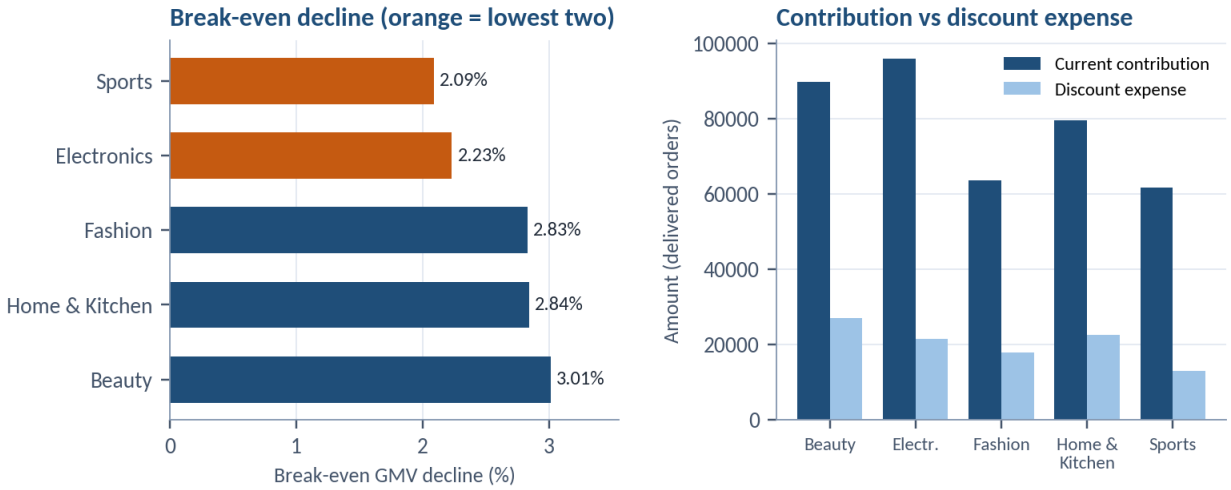


Figure 8 - Left: break-even GMV decline by category (lowest tolerance first). Right: current delivered contribution against discount expense.

What the data shows

- Sports has the smallest tolerance (2.09%), followed by Electronics (2.23%). Beauty has the largest (3.01%). All five categories sit between 2.09% and 3.01%.
- A 10% cut in discount spend saves 1,288.46 in Sports and 2,705.58 in Beauty. Over the five categories the saving is 10,194.57 (sum of the table above).
- Read the percentage as a simple ratio of saving to current contribution, not as a measured volume response (F10).

> **REVIEW FLAGS F1, F10**
>
> The first Q5 query returns NULL for month, so there is no monthly view yet. The break-even ratio does not use the contribution margin on lost sales; check that it matches the definition your case study intends.

SECTION 08

Review Flags and Open Items

Everything that needs a second look before this case study is shared.

Question coverage

Q	Item asked in the question	Status in the source
Q1	Monthly GMV, delivered, cancelled, returned, discounts, refunds, commission, fees, fulfilment, contribution	Query written; result is a single row (month NULL)
Q1	Month-over-month growth; months with GMV up and contribution down	Query written; growth values NULL
Q2	Top 1%, 5%, 10% share of delivered GMV and contribution	Result shown
Q2	Repeat-purchase participation	Result shown (84.51%)
Q2	Customers above 90th percentile with negative contribution	Query written; no result shown
Q2	Are the largest customers also the most profitable?	No written answer; shares are shown in Q2
Q3	Discount rate by category and channel	Result shown
Q3	Higher discount with lower contribution per order	Result shown (risk label)
Q3	Customer segment comparison	Result shown
Q3	High-value transactions with weak economics	Result shown (10 lines)
Q4	Acquisition-month cohorts	Queries written; month NULL
Q4	First-purchase and 90-day cumulative contribution	No query found
Q4	Repeat rate by acquisition channel	No query found (repeat rate is by month)
Q4	Cohorts that worsen by day 90; incomplete 90-day windows	No query found
Q5	Current monthly contribution by category	Query written; month NULL; category totals shown
Q5	Break-even GMV decline; smallest tolerance	Result shown (Sports, then Electronics)

Review flags

Flag	Where	What to check
F1	Q1, Q4, Q5	Month and acquisition-month columns return NULL for every row. Sample dates look like 8/14/2025; check the column type and date format used by DATE_FORMAT and STR_TO_DATE.
F2	Section 02	Reported date range (1/1/2025 to 9/9/2025) is contradicted by sample rows dated 9/30, 12/17 and 12/20/2025. MIN and MAX on text compare characters. True range not confirmed.
F3	Q1	Formula comments describe contribution and seller commission over all rows; the queries apply them to Delivered rows only.
F4	Q2	No result is shown for the large-customer negative-contribution query. The source also repeats the concentration table under the repeat-purchase query.
F5	Q2, Q4	Repeat rate is 84.51% (226 customers with a delivered order) in Q2 and 81.28% (235 customers) in Q4. The base is different.
F6	Q2, Q3, Q4	Order IDs may be shared by several customers: 447 distinct delivered order IDs overall, 721 when counted per customer, and 575 when added across segments.

Flag	Where	What to check
F7	Q3	Distinct delivered customers add to 394 across segments, but only 226 customers have delivered orders; customer_segment may change between rows of the same customer.
F8	Q3	Refund exceeds line GMV on 4 of 10 high-value lines (T00541, T00343, T00433, T00235). Delivered lines carry no refund (Section 02).
F9	Q4	Not found in the submitted queries: first-purchase contribution, 90-day cumulative contribution, channel-level repeat rate, deteriorating cohorts, incomplete-window flags.
F10	Q5	Break-even % = 10% of delivered discount ÷ delivered contribution. It is a simple proxy; confirm it matches the intended definition.
F11	Section 02	Sample row T00002 has an order date (2/12/2025) before its customer acquisition date (5/19/2025).
F12	All tables	Long decimals from the source (for example 298.34000000000003) are rounded to 2 decimals and thousands separators are added. No value was changed.
F13	All tables	Result tables were rebuilt from screenshots. One value (T00235 line GMV) was partly hidden and is read as 1,397.52. The order_id prefix is read as the letter O. Please confirm against the database.
F14	Section 02	Queries use MySQL-style functions; the SQL dialect and the currency are not stated in the source.
F15	Q3	Segment and category tables mix scopes: discount and refund rates use all rows, while orders, customers and contribution use Delivered rows.

What was changed in this edition

- Layout, headings, fonts, table styling, charts and page numbers were redesigned.
- SQL text was kept as written; only line breaks and indentation were tidied.
- Question texts were changed from capital letters to normal sentence case.
- Result tables were retyped from screenshots and given thousands separators; no number was edited.
- Chart values, findings and review flags were added from the source numbers; nothing else was added.